



VIT[®]

BHOPAL

PROJECT REPORT

AI-POWERED SMART SURVEILLANCE AND CRIME DETECTION SYSTEM

USING COMPUTER VISION

A project report on automated person detection, tracking and restricted-zone intrusion alerting using YOLOv8, OpenCV, and Stream lit

Course Code: CSE3010

Slot: F11 + F12

Name: Sunny Patel

Reg. No.: 24BAI10976

Acknowledgement

I would like to express my sincere gratitude to my project guide, [Dr. Gaurav Soni], for the invaluable guidance, encouragement, and technical direction provided throughout the course of this project. Their feedback at every stage — from problem formulation to final implementation — was instrumental in shaping this work.

I am also thankful to [Dr. Sidharth Singh Chauhan], Head of the [Computer Science and Engineering (Artificial intelligence and Machine Learning)], and to the faculty members of [VIT BHOPAL UNIVERSITY] for providing the resources, laboratory facilities, and academic environment necessary to carry out this project.

I extend my thanks to my classmates and peers for their helpful discussions and constructive suggestions during the development and testing of this system. Their perspectives helped refine several design decisions described in this report.

Finally, I am deeply grateful to my family for their constant support, patience, and encouragement throughout this endeavour.

Sunny Patel
24BAI10976

Abstract

Traditional CCTV-based surveillance depends heavily on continuous human monitoring of camera feeds, which is tiring, error-prone, and often results in delayed or missed detection of unauthorized activity. This project presents an AI-Powered Smart Surveillance and Crime Detection System that automates this process using computer vision. The system uses a pre-trained YOLOv8 (You Only Look Once, version 8) object-detection model to locate people in each frame of a video stream, and uses YOLOv8's built-in multi-object tracker to assign a persistent identity (ID) to every detected person across frames. A rectangular restricted zone is defined within the frame, and the system continuously checks whether the centroid of a tracked person's bounding box falls inside this zone.

When an intrusion is detected for the first time for a given tracked ID, the system raises an on-screen alert, saves an annotated screenshot as photographic evidence, and appends a timestamped record to a CSV event log. A Streamlit-based dashboard is provided so that security personnel can review logged intrusion events without needing to interact with the underlying computer-vision pipeline directly. The resulting system demonstrates, on a small scale, how modern object-detection and tracking techniques can be combined into a practical, low-cost intrusion-alerting pipeline suitable for use by security teams, educational institutions, offices, malls, and residential communities.

Keywords: Computer Vision, YOLOv8, Object Detection, Multi-Object Tracking, Intrusion Detection, Restricted Zone Monitoring, Streamlit, OpenCV, Smart Surveillance, Person Re-identification, Event Logging.

1. Introduction

1.1 Background

Closed-circuit television (CCTV) is now installed almost everywhere — in offices, campuses, malls, and residential complexes — but the value of that footage depends entirely on someone watching it. A single security guard is rarely able to track more than a handful of screens at once, and attention naturally drops over long shifts, so unauthorized entry into a sensitive area can easily go unnoticed until the footage is reviewed after the fact, if at all.

Advances in deep-learning-based computer vision, particularly fast single-stage object detectors such as the YOLO (You Only Look Once) family, have made it practical to run person detection and tracking on ordinary video streams in close to real time, even on modest hardware. This project applies that technology to build a small, self-contained surveillance-automation pipeline that can detect people, track them across frames, and automatically flag when someone enters a restricted area.

1.2 Motivation

The motivation for this project stems from a simple observation: most of the intelligence needed to automate basic surveillance tasks already exists in freely available, pre-trained computer-vision models. Rather than requiring an organization to invest in expensive proprietary surveillance software, a functional intrusion-detection pipeline can be assembled from open-source building blocks — OpenCV for video handling, Ultralytics YOLOv8 for detection and tracking, and Streamlit for a lightweight review dashboard. Demonstrating this end-to-end, from raw video to an auditable event log, is the central motivation behind the project.

2. Problem Statement

Continuous manual observation of multiple camera feeds is difficult to sustain and frequently leads to delayed detection of suspicious activity. Existing simple motion-based alarm systems cannot reliably distinguish a person from irrelevant movement (such as shadows, swaying trees, or lighting changes), and they provide no persistent identity for the objects they detect, making it impossible to know whether the same person has entered a restricted area more than once.

There is, therefore, a need for a system that can automatically detect people in a video feed, track their movement with a persistent identity, recognize when someone enters an area they are not authorized to be in, and immediately generate a durable record — an alert, a timestamp, and photographic evidence — without requiring a human operator to be watching at that exact moment.

3. Objectives

The specific objectives of this project are to:

1. Detect people in a video stream automatically using a pre-trained deep-learning object detection model.
2. Track each detected person across frames and assign a persistent, unique identity.
3. Define a restricted zone within the camera's field of view and continuously monitor whether tracked individuals enter it.
4. Generate an immediate visual alert the moment an intrusion is detected.
5. Maintain a durable, timestamped audit trail of intrusion events in a CSV log, together with screenshot evidence.
6. Provide a simple dashboard through which the logged events can be reviewed by nontechnical staff.

4. Scope of the Project

The system focuses on surveillance automation using computer-vision techniques applied to a single video stream at a time. It performs person detection and tracking, monitors one configurable restricted zone, logs intrusion events to a CSV file, captures screenshots as evidence, and exposes a dashboard for viewing the intrusion records. The current implementation is intended as a functional proof of concept rather than a production-grade, multi-camera security platform; extensions in that direction are discussed in Chapter 24 (Future Scope).

4.1 Target Users

The system is designed to be broadly useful to any organization that already relies on CCTV monitoring, including:

- Security personnel and control-room operators
- Educational institutions (schools, colleges, campuses)
- Office buildings and corporate premises
- Shopping malls and retail spaces
- Residential communities and gated complexes

5. Functional Requirements

Functional requirements describe the specific features and capabilities the system provides. The project is organized into four major functional modules, each with a clearly defined input/output structure, satisfying the requirement of at least three major functional modules with a logical workflow.

Table 5.1: *Functional Modules with Input/Output Structure*

Module	Input	Output
--------	-------	--------

Person Detection	Raw video frame	Bounding boxes around detected persons
Person Tracking	Frame + previous track state	Bounding boxes with persistent person IDs
Restricted Zone Monitoring & Alerting	Tracked bounding boxes + zone coordinates	On-screen alert when a person's centroid enters the zone
Event Logging & Dashboard	Intrusion events + intrusion_log.csv	CSV row, saved screenshot, and dashboard view of all events

The logical workflow is as follows: the operator launches the detection script against a chosen video source; the system continuously detects and tracks people in the live preview window; whenever a tracked person's position satisfies the restricted-zone condition for the first time, the system raises an alert, saves evidence, and logs the event; the operator (or any other authorized reviewer) can separately launch the dashboard at any time to review the accumulated log.

6. Non-Functional Requirements

In addition to the functional modules above, the system is designed against the following nonfunctional requirements:

Table 6.1: *Non-Functional Requirements and How the System Addresses Them*

Requirement	How the System Addresses It
Performance	Uses YOLOv8n, the smallest and fastest YOLOv8 variant, to keep per-frame inference time low enough for near real-time preview.
Reliability	Checks that the video source opens successfully before processing begins, and exits gracefully with a clear message if it does not.
Usability	Requires only a single pip install and a single script execution; the dashboard presents logged events in a simple, non-technical table view.
Maintainability	Detection, tracking, intrusion logic, and the dashboard are implemented as separate scripts, so each concern can be modified independently.
Resource Efficiency	The nano-sized YOLOv8 model keeps CPU, memory, and (optionally) GPU usage low compared with larger model variants.
Logging / Auditability	Every intrusion event is recorded with a timestamp, a unique person ID, and a corresponding screenshot for after-the-fact review.

7. Literature Survey

Early automated video-surveillance systems relied on classical computer-vision techniques such as background subtraction and frame differencing to flag “motion” in a scene. These methods are computationally cheap but are notoriously unreliable in practice: they cannot distinguish a person from a moving shadow, a swaying tree, or a sudden change in lighting, which leads to a high rate of false alarms and makes them unsuitable for unattended, always-on deployment.

The introduction of convolutional neural networks brought a new generation of object detectors — two-stage models such as R-CNN and Faster R-CNN, and single-stage models such as SSD and the YOLO family — that can classify and localize specific object categories (for example, “person”) with high accuracy. Single-stage detectors like YOLO process an entire image in one forward pass, which makes them substantially faster than two-stage alternatives and well suited to near-real-time video analysis. YOLOv8, released by Ultralytics, is a further refinement of this family that improves accuracy and training efficiency while offering several model sizes (nano, small, medium, large, extra-large) that trade off speed against accuracy, along with a first-class, built-in multi-object tracking mode.

Detecting a person in a single frame is not, by itself, enough to understand behaviour over time — the system also needs to know that the person seen in frame N is the same person seen in frame N+1. This is the role of multi-object tracking algorithms such as SORT, DeepSORT, and ByteTrack, which associate detections across consecutive frames using position, motion, and (in some variants) appearance cues, and assign each object a persistent identifier. Ultralytics' YOLOv8 exposes this capability directly through its `model.track()` API, which is the mechanism this project relies on to assign the unique person IDs used for zone-intrusion logging.

8. Existing System

Surveillance in most small and mid-sized organizations today is handled in one of two ways. The first, and still the most common, is purely manual monitoring: a guard or receptionist periodically glances at one or more CCTV monitors, or footage is only reviewed reactively after an incident has already occurred. The second is simple motion-detection hardware or software, which flags any pixel-level change between consecutive frames as “motion” and triggers a generic alarm or recording event.

Both approaches are still widely deployed because they are inexpensive and require no specialized machine-learning expertise to set up. However, neither approach reasons about what is actually present in the frame — a manual observer is limited by attention span, and a motion-detection system cannot tell the difference between a person, an animal, a passing vehicle, or a change in lighting.

9. Limitations of Existing System

- Operator fatigue: continuous attention to multiple video feeds is not sustainable over long shifts, and lapses directly translate into missed events.
- High false-alarm rate: simple motion-detection systems trigger on shadows, lighting changes, wind-blown foliage, and other irrelevant motion.
- No persistent identity: without object tracking, there is no way to know whether the same person re-entered a restricted area, or how many distinct individuals were involved in an event.
- Delayed response: incidents are frequently discovered only when footage is reviewed after the fact, by which point the opportunity to respond in real time has passed.
- No structured audit trail: raw video recordings are difficult to search; there is typically no timestamped, per-event log that can be quickly scanned or filtered.
- Cannot distinguish object categories: motion-based alarms cannot restrict alerts specifically to people, as opposed to any other moving object in the frame.

10. Proposed System

The proposed system replaces manual observation and generic motion alarms with an automated, AI-based pipeline that specifically reasons about people. A pre-trained YOLOv8 model detects persons in each video frame; Ultralytics' built-in tracker assigns each detected person a persistent ID across frames; a restricted zone is defined as a rectangular region of the frame; and the system continuously checks whether any tracked person's position places them inside that zone. The first time a given tracked ID satisfies this condition, the system raises an on-screen alert, saves an annotated screenshot, and appends a timestamped row to a CSV event log. A separate Streamlit dashboard allows the resulting log to be reviewed without needing to operate the underlying computer-vision pipeline.

Table 10.1: Existing System vs. Proposed System

Aspect	Existing System	Proposed System
Monitoring	Continuous manual observation, or none until reviewed after the fact	Automated, continuous analysis of every frame
Detection basis	Operator attention or generic pixel-level motion	YOLOv8 deep-learning person detection
Identity over time	Not tracked	Persistent per-person ID via built-in tracking
Alerting	Delayed, dependent on an operator noticing, or a generic motion alarm	Immediate, automatic, on the same frame
Evidence	Manual screen-recording review	Automatic timestamped CSV log + screenshot

Review	Scrubbing through raw footage	Dashboard listing of logged events
--------	-------------------------------	------------------------------------

11. Advantages of Proposed System

- Provides continuous, automated monitoring that does not suffer from operator fatigue or attention lapses.
- Detects the specific object category of interest (people) rather than any generic pixel-level motion, reducing false alarms from shadows or lighting changes.
- Produces an objective, timestamped audit trail (CSV log + screenshot) for every intrusion event.
- Built entirely from free, open-source components with a pre-trained model, so no dataset collection or model training is required to get started.
- Modular script structure (detection → tracking → intrusion logic → dashboard) makes the system easy to understand and extend incrementally.

12. System Requirements

12.1 Hardware Requirements

Table 12.1: *Hardware Requirements*

Component	Minimum	Recommended
Processor	Dual-core CPU, 2.0 GHz	Quad-core CPU, 2.5 GHz or higher
RAM	4 GB	8 GB or more
GPU	Not required (CPU inference)	CUDA-capable NVIDIA GPU for faster inference
Storage	2 GB free space	5 GB+ (for videos, screenshots, logs)
Camera / Input	Pre-recorded video file	Live CCTV / webcam feed

12.2 Software Requirements

Table 12.2: *Software Requirements*

Software / Library	Purpose
Python 3.x	Core programming language / runtime
Ultralytics (YOLOv8)	Pre-trained person detection and built-in multi-object tracking
OpenCV (opencv-python)	Video capture, frame processing, drawing overlays, display window

Streamlit	Web-based dashboard for reviewing intrusion logs
Pandas	Reading and displaying the CSV intrusion log
NumPy	Numerical / array operations supporting OpenCV and YOLO
lap	Linear assignment solver used internally by the tracker
supervision	Computer-vision annotation / utility helper library

13. System Architecture

The system is organized into five logical layers, from raw video input through to the presentation of results. Each layer depends only on the layer beneath it, which keeps the design modular and makes individual layers easy to replace or extend independently (for example, swapping the input layer for a live RTSP camera stream instead of a video file).

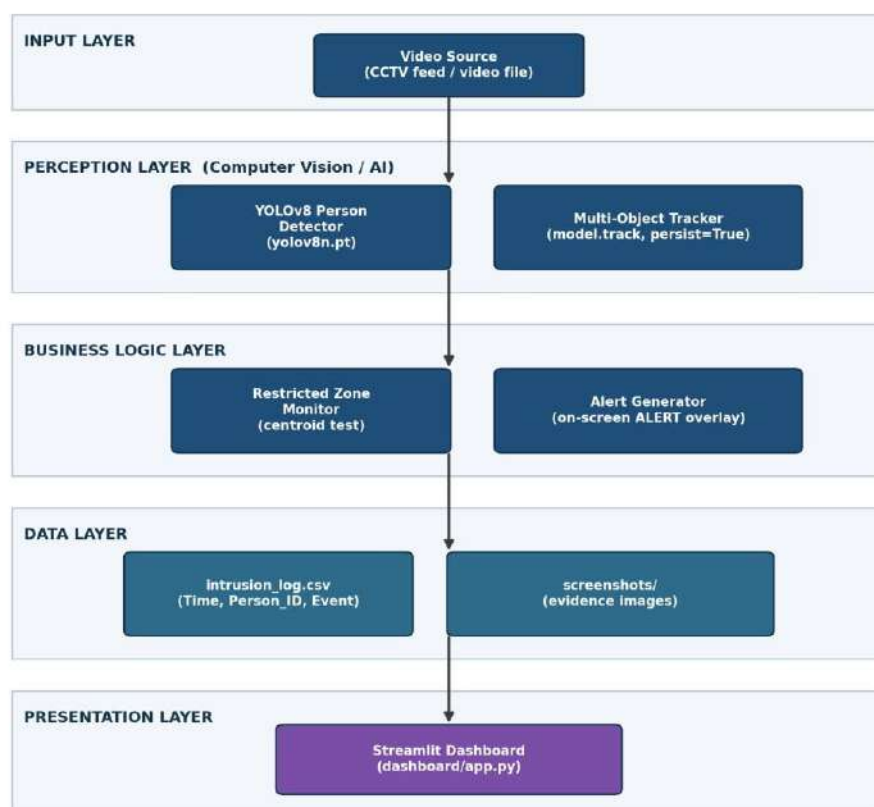


Figure 13.1: Layered System Architecture of the AI-Powered Smart Surveillance and Crime Detection System

Figure 13.1 shows the five architectural layers of the system. The Input Layer supplies raw video frames from a CCTV feed or a recorded file. The Perception Layer applies the YOLOv8 person detector and the built-in multi-object tracker to those frames to produce tracked bounding boxes. The Business Logic Layer evaluates each tracked person's position against the restricted zone and generates an alert when appropriate. The Data Layer persists the resulting events to intrusion_log.csv and saves supporting

screenshots. Finally, the Presentation Layer exposes this stored data to a human reviewer through the Streamlit dashboard.

14. Methodology

The system was developed incrementally, with each script building on the one before it, so that detection, tracking, and intrusion logic could each be verified in isolation before being combined. `detect_people.py` was written first to confirm that the pre-trained YOLOv8n model could reliably detect people in the sample footage. `track_people.py` then layered persistent-ID tracking on top of that baseline using YOLOv8's built-in tracker. Only once both of these behaved correctly was `intrusion_detection.py` written, integrating detection, tracking, a hardcoded restricted-zone rectangle, on-screen alerting, screenshot capture, and CSV logging into a single script. The dashboard was developed last, as an independent, read-only consumer of the CSV log produced by the detection pipeline.

14.1 Key Design Decisions and Rationale

- YOLOv8n (nano) was chosen over larger YOLOv8 variants to prioritize inference speed, since the target use case is near-real-time video review rather than offline, maximum-accuracy analysis.
- A pre-trained, COCO-based model was used rather than training a custom model, because “person” is already one of the 80 COCO classes, and pre-trained weights remove the need for costly dataset collection and training.
- The restricted zone was implemented as a hardcoded rectangle rather than a configurable polygon, to keep the first working version simple; Chapter 24 (Future Scope) discusses making this interactively configurable.
- Intrusion events are logged only once per tracked ID (via a `logged_ids` set) rather than on every frame a person remains inside the zone, to avoid flooding the CSV log and screenshot folder with redundant entries for the same, ongoing intrusion.
- A flat CSV file was used for event storage rather than a relational database, since the event schema is simple (a single record type with no relationships) and a CSV file can be opened directly by non-technical reviewers in any spreadsheet application.
- Streamlit was selected for the dashboard because it allows a data-reading web interface to be built with a small amount of Python code, without requiring separate front-end (HTML/CSS/JS) development.

15. Workflow

Independently of the layered architecture described in Chapter 13, it is useful to trace the workflow that a single video frame follows as it passes through the pipeline, from acquisition to the point where it may (or may not) result in a logged intrusion event.



Figure 15.1: End-to-End Workflow / Process Flow of a Single Video Frame

Figure 15.1 traces a single frame's journey through the system. After a frame is read from the video source, YOLOv8 performs person detection, and the built-in tracker assigns or maintains each detected person's ID. The system then checks whether any tracked person's bounding-box centroid falls inside the restricted zone. If it does, the alert-and-logging stage executes (saving a screenshot and appending a CSV row for any newly detected intrusion), and finally the annotated frame is made available for display and, separately, for later review through the dashboard.

16. Algorithms Used

16.1 YOLOv8 (You Only Look Once, version 8)

YOLOv8 is a single-stage, anchor-free convolutional object detector. Unlike two-stage detectors that first propose candidate regions and then classify each one, YOLOv8 processes the entire image in a single forward pass through its backbone, neck, and detection head, directly predicting bounding boxes, objectness confidence, and class probabilities for a fixed grid of locations. This single-pass design is what makes YOLO-family models substantially faster than two-stage alternatives, at a modest cost in peak accuracy for very small or heavily overlapping objects. Predicted boxes below a confidence threshold are discarded, and Non-Maximum Suppression (NMS) removes duplicate, overlapping boxes for the same object. In this project, the model is restricted to the single COCO class “person” (class index 0), so only human detections are retained from the model's full 80-class output.

16.2 Object Tracking

Detecting a person in isolated frames is not sufficient to reason about behaviour over time; the system must also associate detections of the same person across consecutive frames. This project relies on YOLOv8's built-in tracking mode, invoked via `model.track(frame, persist=True, classes=[0])`. Internally, this associates new detections with existing tracks primarily using positional and motion cues (a Kalman-filter-style motion prediction combined with Intersection-over-Union matching), in the spirit of tracking algorithms such as SORT and ByteTrack. Setting `persist=True` instructs the tracker to retain its internal track state between successive calls, so that a person keeps the same numeric ID for as long as the tracker continues to associate new detections with their existing track.

16.3 Intrusion Detection Logic

The intrusion-detection logic itself is a simple geometric test layered on top of detection and tracking. For each tracked bounding box, the system computes the centroid — the midpoint of the box's x and

y extents — and tests whether that point lies within the restricted zone rectangle, defined by the coordinates (ZONE_X1, ZONE_Y1) and (ZONE_X2, ZONE_Y2). A `logged_ids` set records which tracking IDs have already triggered an intrusion event during the current session, so that the screenshot-andlog action fires exactly once per person, on their first entry into the zone, rather than repeatedly for every frame they remain inside it. The pseudocode below summarizes the logic:

```

for each tracked person box:
    center_x, center_y = midpoint of box
    person_id = tracker-assigned ID

    if ZONE_X1 < center_x < ZONE_X2 and ZONE_Y1 < center_y < ZONE_Y2:
        show 'ALERT: INTRUSION DETECTED' on frame
if person_id not in logged_ids:
    logged_ids.add(person_id)
    save screenshot as screenshots/person_<person_id>.jpg
append (timestamp, person_id, 'Intrusion') to intrusion_log.csv

```

17. System Implementation

This chapter describes the implementation of each functional module in turn, referencing the actual source files in the project repository. Every module is presented alongside a figure placeholder marking where a screenshot of that module's live output should be inserted.

17.1 Person Detection Module

Implemented in `detect_people.py`, this module loads the YOLOv8 nano weights (`yolov8n.pt`) and, for every frame read from the input video, runs a forward pass of the model. For every detected box whose class index equals 0 (the “person” class in the COCO dataset that YOLOv8 is pre-trained on), the module draws a bounding box and a “Person” label, increments a per-frame counter, and overlays the running person count in the top-left corner of the frame.

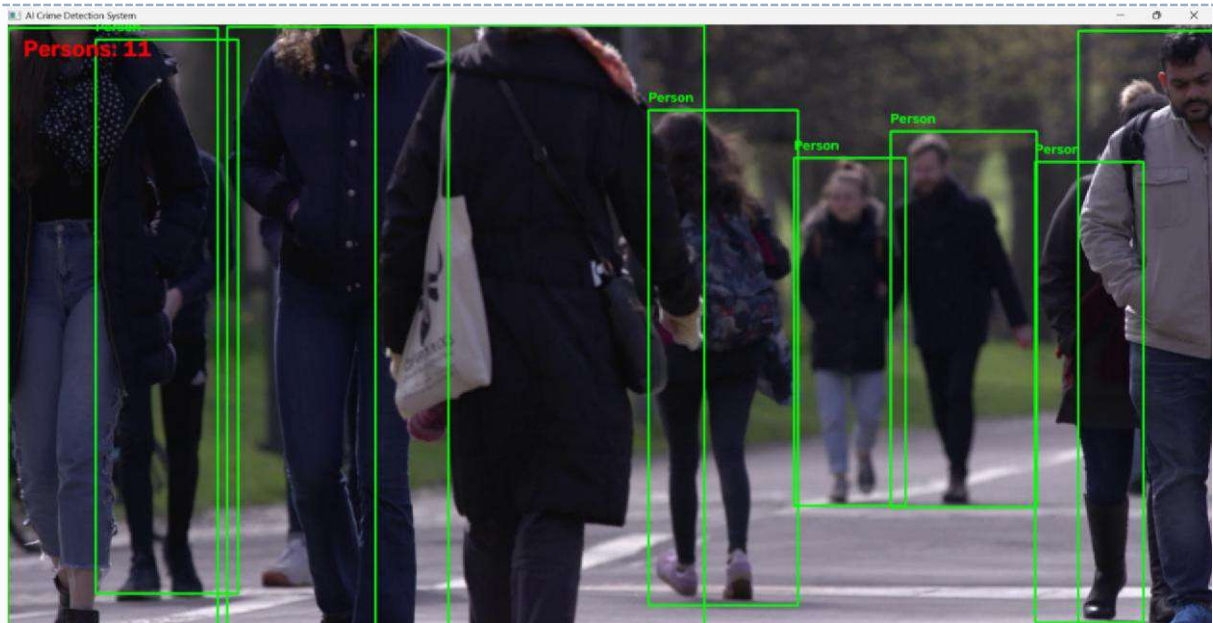




Figure 17.1: *Person Detection Output Showing Bounding Boxes on Detected Persons*

Figure 17.1 is expected to show a frame from the test video with a bounding box and “Person” label drawn around each detected individual, together with the running person count overlaid in the corner of the frame. Replace this placeholder with a screenshot captured while running `detect_people.py` (or the detection stage of `intrusion_detection.py`).

13

17.2 Tracking Module

Implemented in `track_people.py`, this module replaces the plain detection call with `model.track(frame, persist=True, classes=[0])`, which activates Ultralytics' built-in multi-object tracker. Setting `persist=True` tells the tracker to keep matching detections to existing tracks across successive calls (i.e. across video frames) rather than starting fresh each time, so each person is assigned a stable numeric ID that is preserved as long as tracking does not lose them. The `results[0].plot()` call then renders the frame with boxes and ID labels automatically.



15

Figure 17.2: *Person Tracking Output Showing Persistent Tracking IDs*

Figure 17.2 is expected to show the same scene as Figure 17.1, but with each bounding box additionally labelled with its persistent tracking ID (for example, “ID 1”, “ID 2”), demonstrating that the same person retains the same ID across consecutive frames.

17.3 Restricted Zone Monitoring Module

This is the core addition made in `intrusion_detection.py`. The module defines a restricted zone as an axis-aligned rectangle using four hardcoded pixel coordinates, draws that rectangle (with a label) onto every frame, and computes the centroid of each tracked person's bounding box, marking it with a small circle. It then tests whether the centroid falls inside the rectangle to decide whether that person is currently inside the restricted zone.

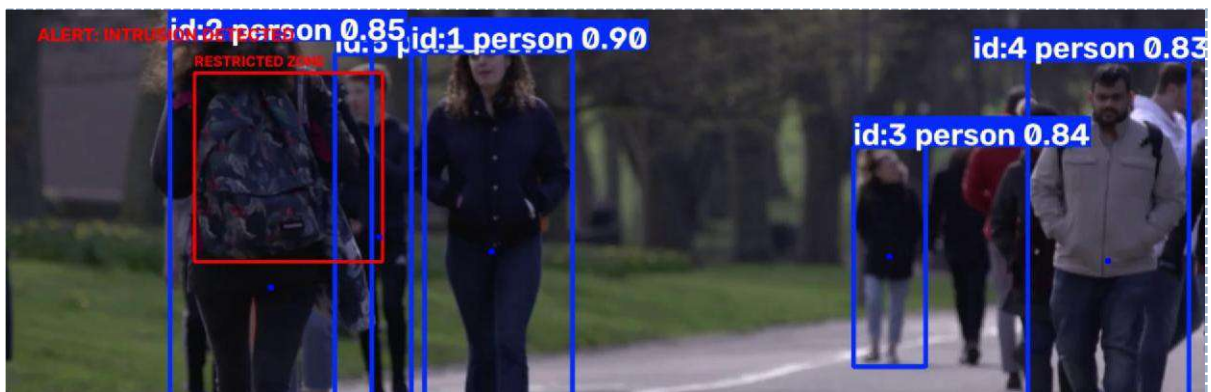


Figure 17.3: *Restricted Zone Overlay Drawn on the Video Frame*

Figure 17.3 is expected to show the restricted-zone rectangle drawn on the video frame, together with



a tracked person's centroid marker, illustrating how the system visually communicates the monitored area to a viewer of the live preview window.

17.4 Alert Generation Module

When a tracked person's centroid is found to lie within the restricted zone, the module overlays a prominent, high-visibility "ALERT: INTRUSION DETECTED" text directly onto the video frame using OpenCV's text-drawing functions, so that anyone watching the live preview window is immediately made aware of the event.



Figure 17.4: On-Screen “ALERT: INTRUSION DETECTED” Overlay

Figure 17.4 is expected to show the moment a tracked person's centroid enters the restricted zone, with the “ALERT: INTRUSION DETECTED” text clearly visible on the frame.

17.5 Event Logging Module

The first time a given tracking ID is found inside the restricted zone, the module (a) saves the current annotated frame as an evidence screenshot to `screenshots/person_<ID>.jpg`, and (b) appends a new row — Time, Person_ID, Event — to `intrusion_log.csv`. A `logged_ids` set in memory ensures this happens only once per person per session, avoiding duplicate entries while that person remains inside the zone. The relevant logic is reproduced below:

```
if person_id is not None and person_id not in logged_ids:
    logged_ids.add(person_id)
    current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    cv2.imwrite(f'screenshots/person_{person_id}.jpg', annotated_frame)
    writer.writerow([current_time, person_id, 'Intrusion'])
```

IntrusionEvent (intrusion_log.csv)	
Time	string (YYYY-MM-DD HH:MM:SS)
Person_ID	integer (tracker-assigned ID)
Event	string (e.g. "Intrusion")

One row is appended per unique intrusion (each Person_ID is logged only once per run).
No relational database or foreign-key relationships are used.

Figure 17.5: Data Model / Record Structure of intrusion_log.csv

Figure 17.5 shows the record structure written to intrusion_log.csv. Because the system uses a flat CSV file rather than a relational database, there is a single entity (IntrusionEvent) with three fields — Time, Person_ID, and Event — and no foreign-key relationships to represent.

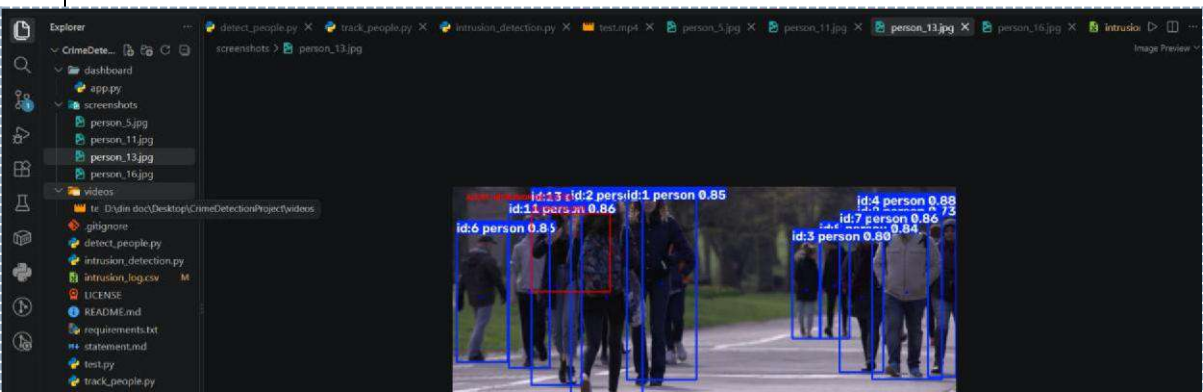


Figure 17.6: Sample Evidence Screenshot Saved in the screenshots/ Folder

Figure 17.6 is expected to show the contents of the screenshots/ folder after a test run, or one representative saved screenshot, demonstrating the photographic evidence captured automatically at the moment of intrusion.

17.6 Dashboard Module

Implemented in dashboard/app.py, this module provides a lightweight Streamlit front end onto the intrusion log. It loads intrusion_log.csv using Pandas and presents the recorded events in a simple table view, so that security staff can review who was flagged and when without needing to run or understand the underlying computer-vision pipeline.

Time	Person_ID	Event	Score	Intrusion
2020-05-12 14:55:21	11	Intrusion	0.86	True
2020-05-12 14:55:49	11	Intrusion	0.86	True
2020-05-12 14:56:09	9	Intrusion	0.86	True
2020-05-12 14:56:14	11	Intrusion	0.86	True
2020-05-12 14:56:21	11	Intrusion	0.86	True
2020-05-12 14:56:37	16	Intrusion	0.86	True
2020-05-12 15:13:41	9	Intrusion	0.86	True
2020-05-12 15:13:48	11	Intrusion	0.86	True
2020-05-12 15:13:57	23	Intrusion	0.86	True
2020-05-12 15:14:03	16	Intrusion	0.86	True
2020-05-12 15:14:08	9	Intrusion	0.86	True
2020-05-12 15:14:14	11	Intrusion	0.86	True
2020-05-14 18:24:17	11	Intrusion	0.86	True
2020-05-14 18:24:24	23	Intrusion	0.86	True
2020-05-14 18:24:39	16	Intrusion	0.86	True
2020-05-14 18:24:43	9	Intrusion	0.86	True
2020-05-14 18:25:19	11	Intrusion	0.86	True
2020-05-14 18:25:37	23	Intrusion	0.86	True
2020-05-14 18:26:41	16	Intrusion	0.86	True
2020-05-14 18:30:06	9	Intrusion	0.86	True
2020-05-14 18:30:18	11	Intrusion	0.86	True
2020-05-14 18:30:18	23	Intrusion	0.86	True
2020-05-14 18:30:34	26	Intrusion	0.86	True

Figure 17.7: Streamlit Dashboard Displaying Logged Intrusion Events

Figure 17.7 is expected to show the Streamlit dashboard opened in a browser, displaying the intrusion_log.csv records in tabular form, giving a reviewer an at-a-glance summary of all logged intrusions.

18. Results and Discussion

When run against a sample video, the system reliably detected people present in frame, drew consistent bounding boxes, and preserved a stable ID for each tracked individual across frames under normal conditions. Entry of a tracked person's centroid into the defined restricted rectangle correctly triggered the on-screen alert, and the first such event for each ID produced exactly one screenshot and one corresponding row in intrusion_log.csv, giving a clear, timestamped audit trail.

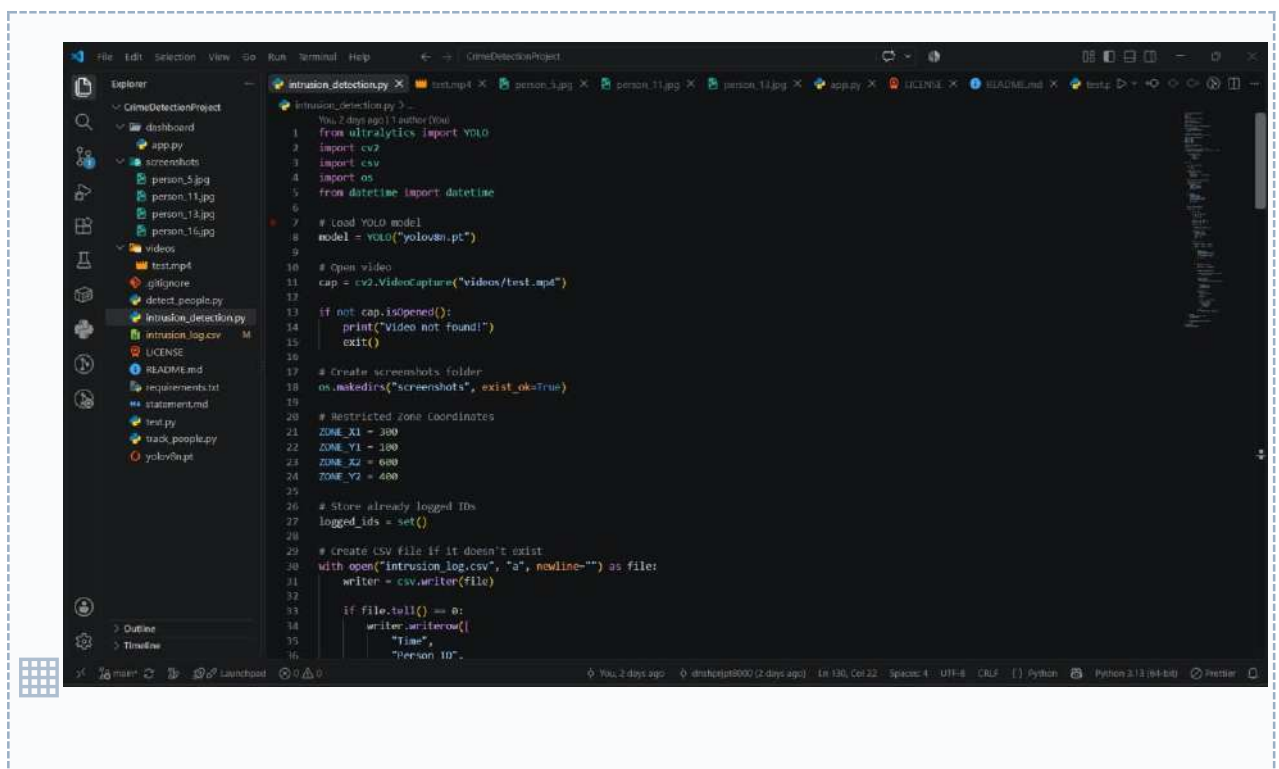




Figure 18.1: Overall System Output on a Sample Test Video (Detection + Tracking + Zone Overlay)

Figure 18.1 is expected to show a composite or representative view of the full pipeline running on the test video — detected and tracked persons, the restricted zone, and (where applicable) an active alert — in a single frame, summarizing the qualitative results described in this chapter.

Because the system uses YOLOv8n — the smallest and fastest variant in the YOLOv8 family — it favours inference speed over peak detection accuracy. This is an appropriate trade-off for near-real-time monitoring on modest hardware, though it means the system is more likely to miss detections in heavily crowded scenes, under poor lighting, or when a person is significantly occluded, compared with larger YOLOv8 variants (s / m / l / x). No formal benchmark (precision, recall, mAP) was computed against a labelled dataset for this project; the evaluation here is qualitative, based on running the pipeline against the test footage included in the repository.

Table 18.1: Sample Excerpt from *intrusion_log.csv* (Illustrative Format)

Time	Person_ID	Event
2026-01-14 10:32:07	1	Intrusion
2026-01-14 10:33:52	3	Intrusion
2026-01-14 10:41:18	4	Intrusion

In the completed report, Table 18.1 should be replaced with an excerpt from the actual intrusion_log.csv generated during a real test run, alongside the corresponding screenshots referenced in Chapter 17.

19. Use Case Diagram

The use case diagram in Figure 19.1 models the system from the perspective of its two external actors — the Security Operator, who starts the system and reviews its output, and the CCTV Camera, which supplies the raw video feed — together with the sequence of automated use cases that the system itself performs on every frame.

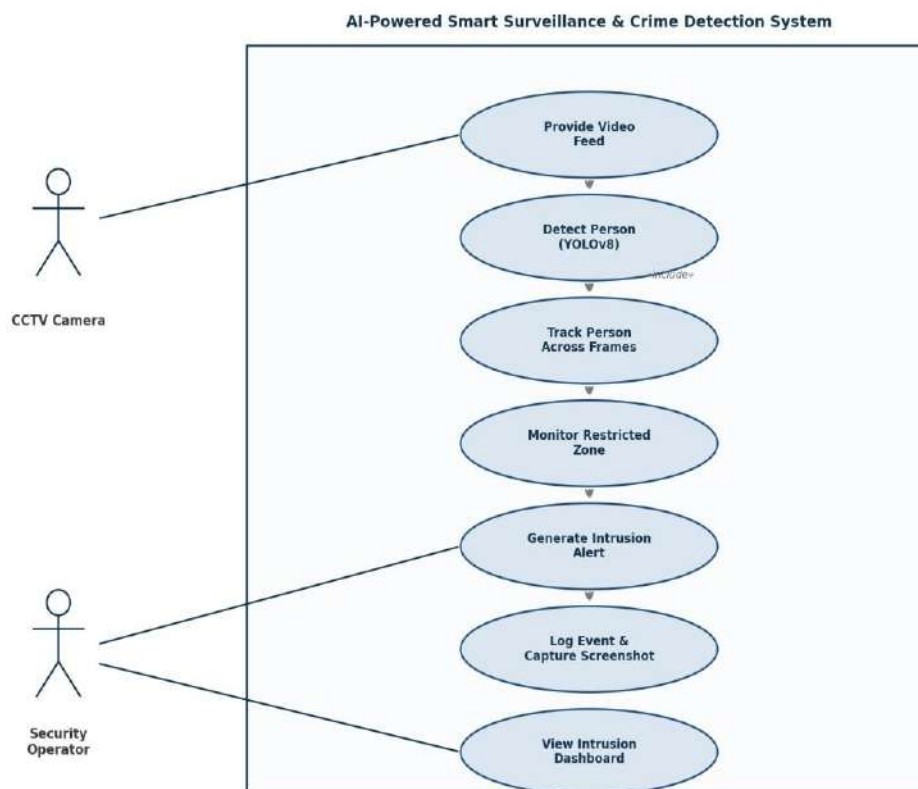


Figure 19.1: Use Case Diagram of the AI-Powered Smart Surveillance and Crime Detection System

As shown in Figure 19.1, the CCTV Camera actor is associated with the “Provide Video Feed” use case, which «include»-chains through Detect Person, Track Person, Monitor Restricted Zone, Generate Intrusion Alert, and finally Log Event & Capture Screenshot — mirroring the automated pipeline described in Chapters 15 and 16. The Security Operator actor is associated with “Generate Intrusion Alert” (as the recipient of the on-screen warning) and with “View Intrusion Dashboard”, which is where the operator reviews the accumulated log at a time of their choosing.

20. Sequence Diagram

The sequence diagram in Figure 20.1 shows the time-ordered messages exchanged between the Security Operator and the system's internal components for one complete cycle: starting the detection

pipeline, processing a frame through to a logged intrusion, and later reviewing that event on the dashboard.

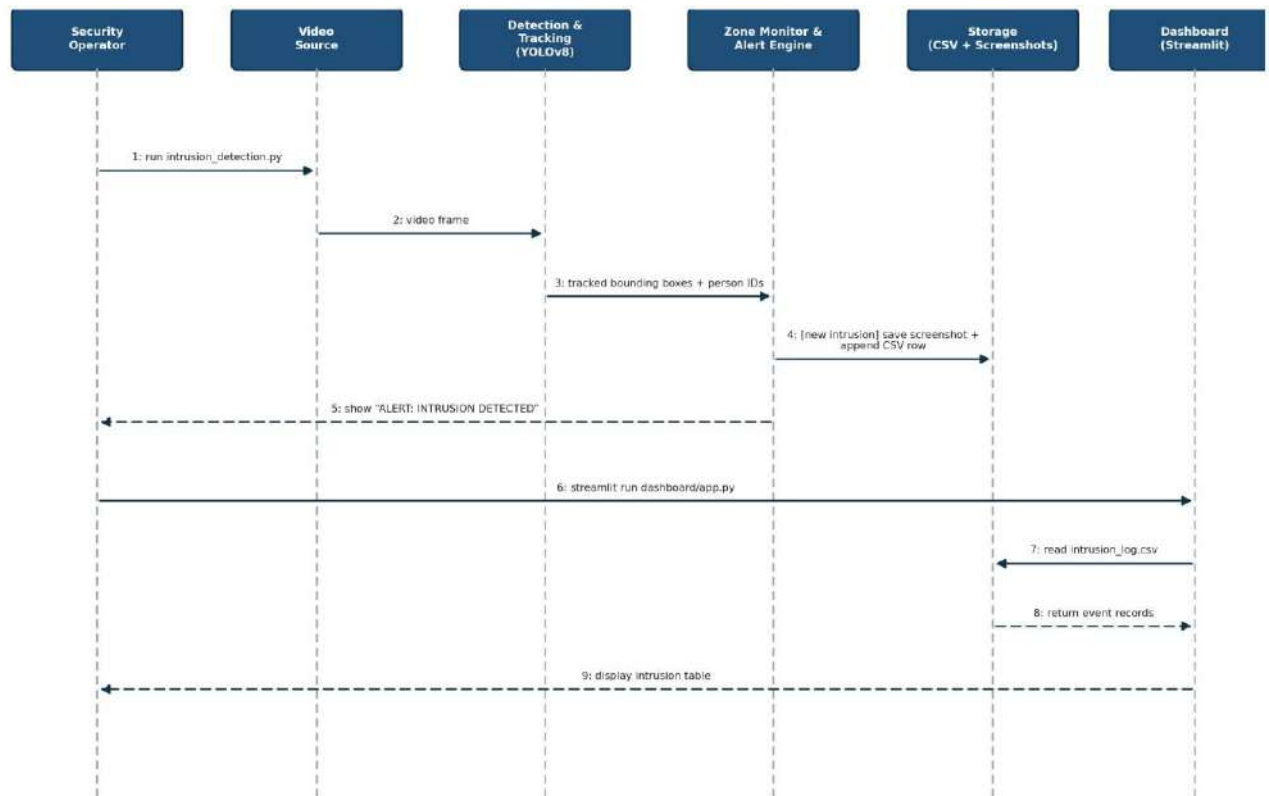


Figure 20.1: Sequence Diagram Showing Interaction Between System Components

Figure 20.1 begins with the Security Operator starting `intrusion_detection.py`, which reads a frame from the Video Source and passes it to the Detection & Tracking component. The resulting tracked bounding boxes and person IDs are passed to the Zone Monitor & Alert Engine, which — for a newly detected intrusion — instructs the Storage component to save a screenshot and append a CSV row, and separately notifies the Operator with an on-screen alert. In a later, independent interaction, the Operator launches the Dashboard, which reads the accumulated records from Storage and displays them as an intrusion table.

21. Class Diagram

The project is implemented as a small collection of procedural Python scripts (`detect_people.py`, `track_people.py`, `intrusion_detection.py`, and `dashboard/app.py`) rather than as a set of custom object-oriented classes; the only classes involved are internal to the third-party Ultralytics library (for example, its YOLO model class) and are not defined within this project's own codebase. A conventional UML class diagram — listing attributes and methods of project-defined classes — would therefore not accurately represent the implementation. In its place, Figure 21.1 presents a component-level design

diagram, which shows the same logical decomposition of responsibilities (what a set of classes would otherwise represent) as discrete components with explicit dependencies between them.

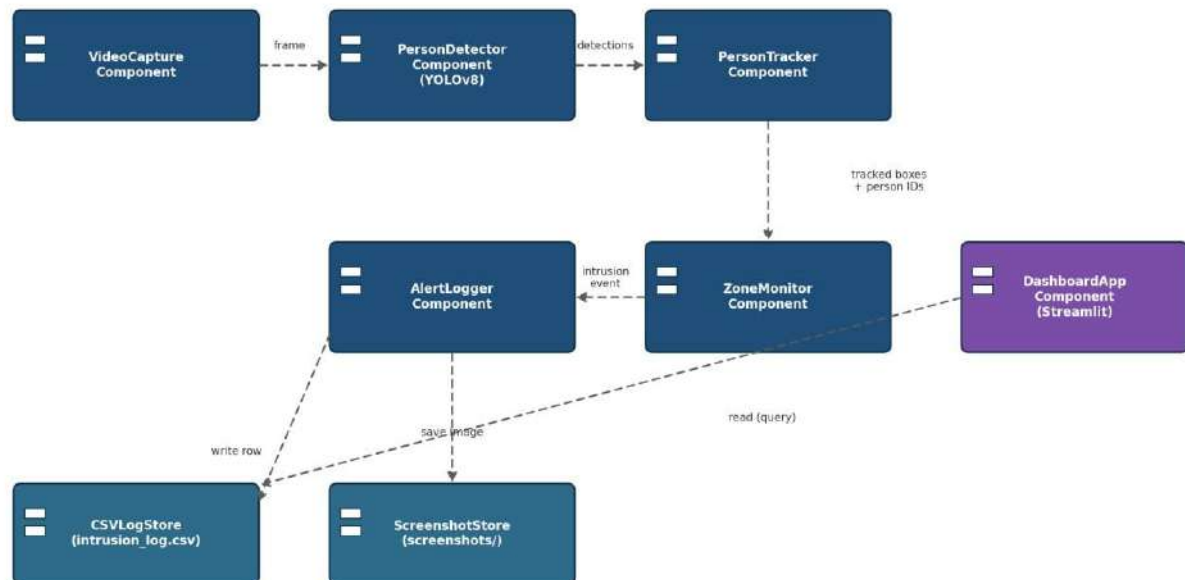


Figure 21.1: *Component-Level Design Diagram (presented in place of a Class Diagram — see note above)*

Figure 21.1 shows eight components: VideoCapture, PersonDetector, PersonTracker, ZoneMonitor, and AlertLogger form the processing pipeline (each dependency arrow is labelled with the data passed between components, e.g. “frame”, “detections”, “tracked boxes + person IDs”); CSVLogStore and ScreenshotStore represent the two persistent data stores written to by AlertLogger; and DashboardApp is shown as a read-only consumer of CSVLogStore, independent of the real-time processing chain.

22. Testing and Validation

Each script was exercised individually before being combined — detect_people.py and track_people.py were run first to confirm that detection and tracking behaved correctly on the sample video, and test.py was used as a scratch script during development. intrusion_detection.py was then tested end to end. The table below summarizes the key manual test cases and their expected results.

Table 22.1: *Test Cases and Expected Results*

ID	Test Case	Expected Result
TC-01	Load yolov8n.pt on startup	Model loads without error

TC-02	Open videos/test.mp4	Video capture opens successfully; program exits with a clear message otherwise
ID	Test Case	Expected Result
TC-03	Person appears in frame	Bounding box and label are drawn around the person
TC-04	Same person across consecutive frames	Person keeps the same tracking ID
TC-05	Person's centroid enters the restricted zone	"ALERT: INTRUSION DETECTED" text is overlaid on the frame
TC-06	First-time intrusion for a given ID	One screenshot saved and one new row appended to intrusion_log.csv
TC-07	Same ID remains inside the zone in later frames	No duplicate CSV row or screenshot is created for that ID
TC-08	Press 'q' during playback	Preview window closes and the program exits cleanly
TC-09	Open dashboard after a session with logged events	Streamlit dashboard displays the contents of intrusion_log.csv

23. Challenges Faced

- Calibrating the restricted-zone rectangle: because the zone coordinates are hardcoded in pixels, they had to be manually tuned to match the framing and resolution of the specific test video, which is not directly portable to a different camera angle or resolution.
- Avoiding duplicate log entries: an early version logged an intrusion on every frame a person remained inside the zone; introducing the `logged_ids` set to record which IDs had already been logged was necessary to produce one clean entry per intrusion.
- Balancing speed against accuracy: choosing YOLOv8n keeps the pipeline fast enough for nearreal-time preview, but required accepting a higher chance of missed detections in more demanding scenes than a larger YOLOv8 variant would provide.
- ID stability under occlusion: when a tracked person is briefly hidden behind another object or person, the tracker can lose and later reassign a new ID to the same individual, which could in principle cause them to be logged more than once.
- Keeping the pipeline responsive while writing files: saving a screenshot and appending a CSV row on every new intrusion had to be done without noticeably stalling the live video preview loop.

24. Future Scope

- Allow the restricted zone to be drawn interactively (e.g. on the dashboard) instead of hardcoding pixel coordinates in the source code.
- Extend the pipeline to support multiple camera feeds concurrently.
- Integrate real-time notifications via email, SMS, or messaging platforms (e.g. Telegram) when an intrusion is logged.
- Move logs and screenshots to cloud storage or a proper database for scalability and remote access.
- Add face recognition or person re-identification to flag previously identified individuals across separate sessions.
- Evaluate and, if needed, fine-tune a larger YOLOv8 variant (or a custom-trained model) for higher accuracy in the target deployment environment.
- Package the system for containerized deployment and add richer dashboard analytics such as footfall trends and heatmaps.

25. Conclusion

This project demonstrates that a practical, automated intrusion-alerting system can be built from freely available computer-vision tools without the need for custom model training. By combining YOLOv8's pre-trained person detector with its built-in multi-object tracker, the system is able to detect people, follow them across frames, recognize when they enter a defined restricted zone, and automatically produce a timestamped, evidence-backed log of that event — all with a relatively small, modular Python codebase. While the current implementation has clear limitations around configurability, multi-camera support, and external alerting, its modular design (separate detection, tracking, intrusion-logic, and dashboard components) provides a solid foundation on which those enhancements can be built. Overall, the project shows how modern object-detection and tracking techniques can meaningfully reduce the reliance on continuous human monitoring of CCTV feeds.

26. References

1. Ultralytics YOLOv8 Documentation — <https://docs.ultralytics.com>
2. OpenCV Documentation — <https://docs.opencv.org>
3. Streamlit Documentation — <https://docs.streamlit.io>
4. COCO: Common Objects in Context Dataset — <https://cocodataset.org>
5. Bewley, A. et al., “Simple Online and Realtime Tracking (SORT)”, 2016.
6. Zhang, Y. et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box”, 2022.
7. Project Repository — <https://github.com/dnshprjpt8000/AI-Crime-Detection-System>